

Open Research Project

VerifiQuant

**Don't trust the answer.
Verify the failure.**

R14946005 柯宥圻

AI 的生產力幻覺

真實案例

Dustin (台大財金同學)

J.P. Morgan 投行分析師

- 每週工時 80 小時
- 肉眼從數百頁 PDF 財報中找數字
- 手動貼進 Excel 反覆驗證
- 88% 的試算表腳本隱藏人為錯誤

「我寧可自己算，也絕對不敢用 AI。」

KPMG 全球調查

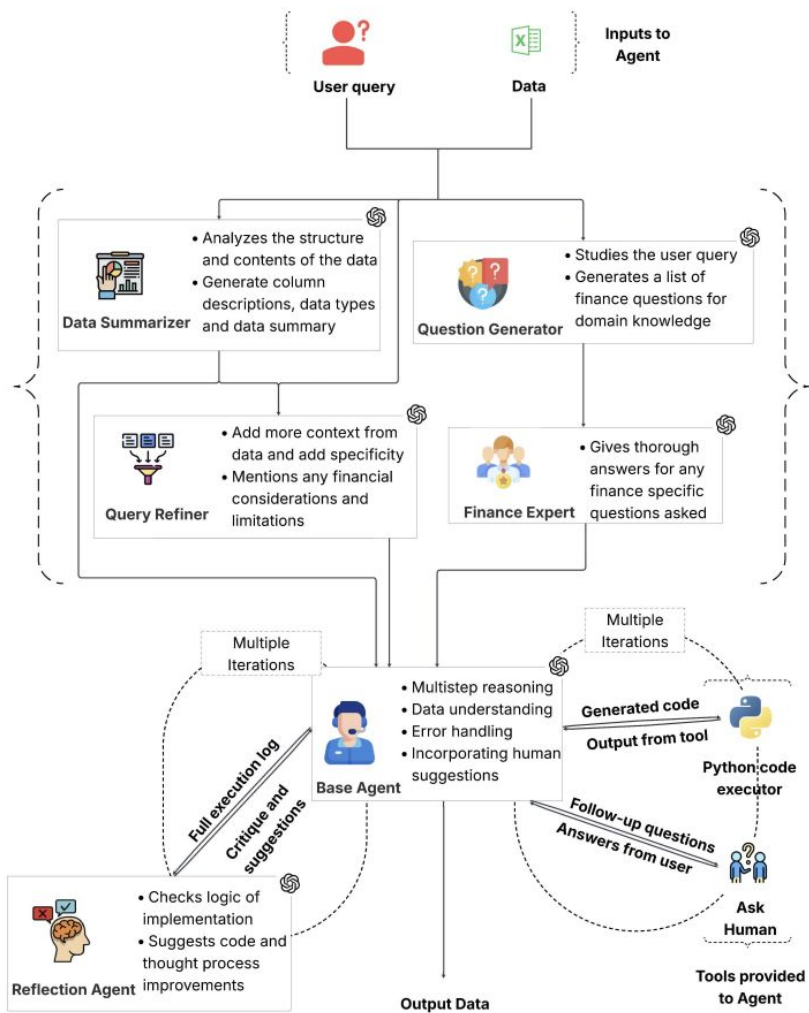
74%

對導入 AI 的高度關注

11%

真正投入生產部署

「人人都渴望 AI 的效率，卻沒人敢承擔 AI 幻覺的代價。」



JP Morgan 在 2025/08寫的multi-agent paper

(EMNLP 2025)

46%

回答一次就正確的機率

就算這是 SOTA
你會用他嗎？

Framework	Pass@1	Pass@2	Pass@5	Pass@10
Base-Agent	39.59	45.55	51.85	55.0
Base-Agent + Reflection	39.00	46.62	53.13	56.0
Base-Agent + Insights	43.89	53.84	64.87	72.0
Multi-Agent	46.00	57.16	67.69	72.0

「AI 馬屁精」與致命的靜默錯誤

10% AI 隨機靜默出錯

=

100% 題目須全部人工重查

SILENT FAILURE 靜默錯誤

AI 遇到沒有明文規定的模糊地帶時

為了討好你、急著給答案，會「假裝自己懂」，替你預設條件，再極度自信地給出落差數百萬美元的錯誤數字。

從CoT到Multi-Agent，模型往往會「掩蓋真正的錯誤原因」。

16% 歧意率

ACL2025 論文指出：財金資料集本身充滿歧義

舉例：期初 vs 期末付款：隱性共識

雙方的默契往往不寫進白紙黑字，AI 一旦自行預設和使用者的假設不同的情況，即引發蝴蝶效應。

「90% 準確率」在金融業是假命題。

純粹 Prompt Engineering 的三宗罪

LLM 在金融 benchmark 的準確率已逼近頂峰，但 accuracy 本身在實務部署中是誤導性指標。

1. 如果模型遇到不確定性，如何反應？
2. 如果數字會隱性算錯，系統還有用嗎

1

Silent Failure

靜默失敗

模型在資訊不足時，仍以高自信輸出答案。

2

Semantic Drift

語意漂移

公式選對，但隱性歧義(匯率方向、現金流期初／期末、報酬率年化基準)導致系統性錯誤。

3

Audit Gap

審計斷層

CoT 軌跡因取樣隨機性無法重現，損失發生時無從究責。

法規現實與我們的護城河

台灣 FSC 金管會 第四款

人類可控

金融員工必須承擔最終責任，
且 AI 決策過程必須可解釋。

歐盟 EU AI Act –rule 13

精確追溯

部署者必須能夠解釋模型如何得出結論，
並必須能將該結論精確追溯至原始的輸入紀錄。

Basel III & FINRA - 2025

輸出下限 + 人為檢查點

建立透明AI框架時，需實施HITL，
並在必要時對高風險應用設定「輸出下限」作為安全防護。

監管巨頭只問一件事：出事了，能不能精確追溯？

✗ Multi-Agent (黑箱共識)

讓多個黑箱互相聊天達成共識。出錯時只能交出神經網路「事後合理化」的對話紀錄，在法庭上毫無審計效力。

✓ VerifiQuant (合約驗證)

明確的人為檢查點 HITL、可證明的追溯性，溯源到輸入內容，以及 AI 不懂時的「輸出下限」，完全對齊監管要求，且不受 CoT 隨機性干擾，這就是我們的護城河。

VerifiQuant: 合約驗證式金融推理框架



AI 的工作不是算數學, 而是「填寫合約」。

6 層診斷漏斗 (M · N · F · E · I · C) — 算出任何數字前, 資料必須逐層通過:



任一閘門未通過 → VerifiQuant 不瞎猜, 強制暫停、發出結構化警告, 將決策權交還人類。

價值主張：用「安全拒答」換取 絕對信任

	Gemini 2.5 單次輸出	最強 CoT + 盲審 (Pro 3) + 多角度自我檢查 ×3	VerifiQuant
準確率	82%	98%	90%
靜默錯誤	18%	2%	0%
	風險最高	消除不掉的致命風險	完美攔截
安全拒答	0%	0%	10%
平均時間	7 secs	3 mins	26 secs

我們寧可打斷分析師、請他再確認，也絕不給出不可審計的數字。

把不可靠的高準確率，換成金融界最需要的 100% 絕對信任。

www.verifiquant.com
demo網址

Demo: 讓合規變得一目了然

真正的透明度，不只是丟給法遵人員模型能力圖和答案，而是看見每一次決策的依據。

接下來 4.5分鐘實機展示：VerifiQuant 如何攔截致命的隱性假設，並 產出符合 EU AI Act 級別的審計報告。

附錄

原始研究進度報告 — 完整技術細節、實驗設計與佐證資料

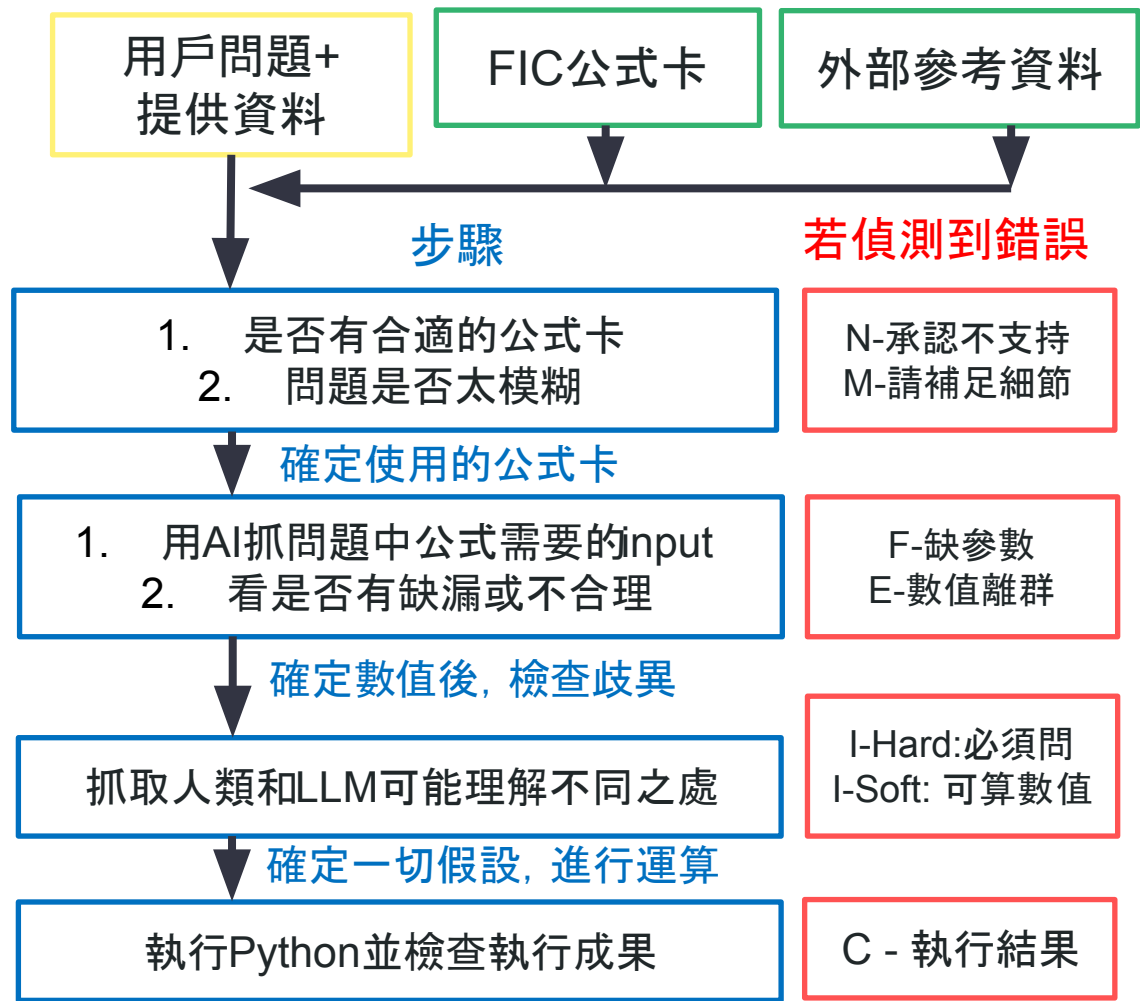
CoT 的 Faithfulness 危機，導致AI的trace無法復現+虛假信心

Jacovi & Goldberg (ACL 2020) 區分了兩個常被混淆的性質：

1. **可信度 (plausibility)**——解釋對人類觀察者是否具有說服力
2. **忠實度 (faithfulness)**——解釋是否精確反映模型內部的真實推理過程。

最危險的情境是高可信度、低忠實度的組合：**一個看起來很有說服力的解釋，實際上完全不對應模型的內部決策。**

Turpin et al. (NeurIPS 2023) 透過輸入操弄實驗證明，CoT 中間步驟並不必然反映模型實際得到答案的計算路徑——**模型可能受隱性偏見影響，但 CoT 軌跡卻不揭示此影響。**



1. 系統抓取FIC與外部資料

2. 檢查抓取的外部資料
是否和問題直接相關

3. 驗證input是否有空
缺或不合理之處

4. 評估是否有可能對最終
數值產生影響的模糊之處

5. 嘗試執行code, 檢查
是否運行成功

核心設計：分類漏斗 (Diagnostic Funnel) 錯誤類型說明

1	Intent Misunderstanding	語義模糊，意圖太籠統無法對應 FIC	Refusal + 引導式反問
2	Scope Not Supported	意圖明確，但 FIC 庫中無對應公式	Graceful Exit: 宣告知識邊界
3	Schema Formula Spec	FIC 已選定，但缺少必要參數	Slot-filling: 動態補值
4	Boundary Extraction Alert	數值存在但違反邏輯邊界	Deterministic Alert: 要求確認
5	Critic Interception	隱性語義歧義 (年末/年初、匯率方向)	I-hard 阻止 I-soft 附警告
6	Logic Calculation	確定性執行: C-exec / C-match / C-mismatch	程式碼執行 → 歸因分析

I類檢測成功且重要範例 1 :TRIN公式

$$TRIN = \frac{\text{Advancing Stocks} / \text{Declining Stocks}}{\text{Advancing Volume} / \text{Declining Volume}}$$

問題:主觀閾值判斷

TRIN 公式固定, 但「市場情緒」是主觀標籤。

TRIN < 1.0 通常看多, 但有交易員把 0.5 以下才視為極度超買。**期望輸出為布林 值(true/false)**, 而非數值計算。

✓ 審計邏輯

如果框架不攔截, LLM 會隨機根據訓練資料選一個閾值, **產生「隱性假設」**。框架強迫用戶(或系統預設)定義這個閾值, 確保審計一致性。

關鍵價值: 將「隱性主觀假設」轉為「顯性可審計參數」, 消除LLM 隨機閾值選擇問題

I類檢測成功且重要範例 2 :公式歧義

⚠ 現實狀況: 題目同時給了 `current_liabilities` 和 `total_debt`, LLM 會隨機挑公式

公式 A (Ground Truth 採用)

$$\frac{\text{Total Tangible Assets}}{\text{Total Debt}}$$

VS

公式 B (模型可能選擇)

$$\frac{(\text{Total Tangible Assets} - \text{Current Liabilities})}{\text{Long-term Debt}}$$

VerifiQuant 的價值: 框架偵測到「數據可算兩種答案」→ 語義歧義 (Semantic Ambiguity)

✓ 解法: 主動詢問用戶選擇公式及對應 code, 消除歧義

「數值不同但推理正確」→ 非模型錯誤, 是需要人類決策介入的問題

Demo 演示

📄 示範題目 (NPV 計算)

Context: 一家新創科技公司正在考慮投資一個新的軟體開發專案。初始投資 10 萬美元(第一年年初)。未來五年現金流: 2萬, 3萬, 3.5萬, 4萬, 4.5萬。折現率 8%, 年底到款。

Question: 該軟體開發專案的淨現值 (NPV) 是多少? 答案取最接近的整數。

✓ 正確答案: \$32,050 (NPV)

同一題目, 透過 6 種不同的提問方式, 展示 VerifiQuant 對各種錯誤類型的處理能力。

Case 1 正常詢問

✓ 成功

精確提問 → 找到卡片 →
正確計算 \$2,940

Case 2 模糊詢問

N 類錯誤

只告訴他「我想賺波大的」
→ 未匹配NPV的 FIC 卡片
→ 跟用戶說RAG失敗

Case 3 模糊詢問2

M 類錯誤

告訴他「我想賺錢」→ 找到
NPV和IRR的 FIC 卡片 →
反問用戶啥是「賺錢」

Case 4 缺少關鍵參數

F 類錯誤

忘記提供缺少折現率 →
Slot-filling 補值

Case 5 數值不合邏輯

E 類錯誤

初始投資抓成負值 → 自動
提醒確認

Case 6 模糊時間條件

I 類錯誤

刪除「年末」→ 系統攔截 →
詢問年初或年末

CoT 的 faithfulness 進一步討論，看怎麼毒害結果

Lanham et al. (2023) 量化了 CoT 步驟與最終答案的因果 脫節，並揭示一個反直覺的 scaling 趨勢：隨著模型參數規模擴大與能力增強，其生成的推理軌跡反而越不忠實。**模型越聰明，其偽裝推理軌跡以符合人類期待的能力就越強**。

Pfau et al. (ICLR 2024) 從機制層面提供了解釋：**過去假設 CoT 的有效性來自將複雜任務拆解為人類可讀的離散步驟**，但嚴格實驗證明 CoT tokens 提供的是純粹的計算深度 (computational depth)，而不需要包含任何有意義的中間語義資訊 —— 甚至將中間步驟替換為無意義的填充符號也不影響效能。

近期paper (Lie to Me, 2026; Ariadne, 2026) 將此分析延伸至 reasoning models 與 multi-agent 系統，**確認此現象並非早期模型的暫時性缺陷**。

現有解法的光譜與缺口

- (a) 執行端外部化 (PAL, PoT, FINDER) 將算術交給 Python interpreter, 但 LLM 仍可寫出語法正確、金融邏輯錯誤的程式碼, 對 semantic drift 無防禦;
- (b) 符號求解器路線 (Logic-LM, VERAfi) 依賴自然語言到形式邏輯的脆弱翻譯, 在複雜情境下準確率反而劣化;
- (c) Agentic reflection (ToT, Self-Refine) 以 LLM 驗證 LLM, 未解決 faithfulness 的本質問題。

最常被討論的一點是方法(b)符號求解器路線

Neuro-symbolic 與 Formal Verification

為取得formal保證，過往研究嘗試讓 LLM 充當「翻譯者」，**將自然語言問題轉換為確定性符號系統可處理的表示**。此流派可拆為兩條子路線，但都最終受限於同一個根本瓶頸。

子路線1: 模組化推理 - DeepMind

借鑑 neurosymbolic 概念, 提出 Selection-Inference (SI) 框架, 將推理拆為「**從文本選取相關事實**」與「**從事實推導結論**」兩個獨立模組。後續的 Faithful Reasoning 進一步加入 Halter 模組(偵測「**資訊是否已足夠**」)與 Value Function + Beam Search (並行推演多條邏輯分支後選優)。

這兩篇工作在可解釋性上取得顯著進展, 並明確批評 CoT 等方法以「幻覺合理化」冒充忠實推理。然而, 其表現的前提假設始終未能突破:**所有必備前提、規則與干擾項必須事先完整封裝在 Context 中(我們做不到!)**。一旦需要外部檢索, Selection 模組若抓取錯誤的前提, 後續推論無論邏輯多完美, 皆只是在錯誤的基礎上進行華麗的推導。

子路線2：形式求解器路線

Logic-LM (Pan et al., 2023) 讓 LLM 將題目翻譯成 ASP, 交由 Clingo 這類確定性 Symbolic Solver 驗證。

LINC (Olausson et al., EMNLP 2023) 嘗試映射至一階邏輯(FOL), **但其錯誤分析明確指出模型頻繁出現語法錯誤與語意偏移(Semantic Drift)**

SATLM (Ye et al., NeurIPS 2023) 改採宣告式路線, 讓 LLM 只生成 Z3 SMT Solver 的約束式而非直接解題, **但仍高度依賴特化 Few-shot Prompt 才能驅動**, 且剛性的 SMT Solver **對自然語言中的雜訊零容忍**, 一個符號錯誤即導致 UNSAT 或 Crash, 且架構為單向 Parse→Solve, 無自我修正機制。

Logic-LM 即使加上自我修正模組, 在複雜邏輯資料集(AR-LSAT)上的準確率仍僅達 43.04%。VERAFI 則在失去翻譯精度後退而求其次, 改採「軟約束」(之前提到的只是把約束放在prompt裡面祈禱ai會去主動注意到), 形式保證大幅削弱

翻譯行為的根本極限

最新的 FormalJudge (Zhou et al., arXiv 2026, under review) 在其系統的限制分析中指出：**將自然語言需求翻譯為形式化規範，在一般情況下是一個「不可解的問題 (undecidable problem)」，並將此現象命名為「規格合成瓶頸 (specification synthesis bottleneck)」。**

即便 FORMAL JUDGE 採用雙向架構——將使用者意圖分解為原子級「是/否」問題後由 Z3 組合——其失敗案例分析仍顯示：剩餘錯誤主要集中於意圖分解與語意提取這兩個翻譯環節，並結論「**這是一個單靠形式化驗證也無法解決的問題**」。

Multi-Agent Systems 與 Self-Reflection

以 multi-agent 協作或 self-reflection 提升可靠性是目前金融 agent 最常見的商業路線 (ToT, Self-Refine, Reflexion)。然而這條路線在 faithfulness 上存在本質缺陷：**CoT 與自我驗證的本質是「因果 脫節 (Causal Disconnect) 的事後合理化」——LLM 宣稱已完成驗證，實際上是根據最終答案反向生成了一段看起來合理的推理軌跡。**

在 multi-agent 架構中，這一問題被放大為 theory-of-mind failure: Why Multi-Agent LLM Systems Fail (Cemri et al., ICLR 2026) 指出，MAS 最常見的失敗模式在於 agent 之間假設彼此理解，但對共享狀態的詮釋存在分歧，導致錯誤在 agent 邊界處被靜默傳遞而非攔截。

